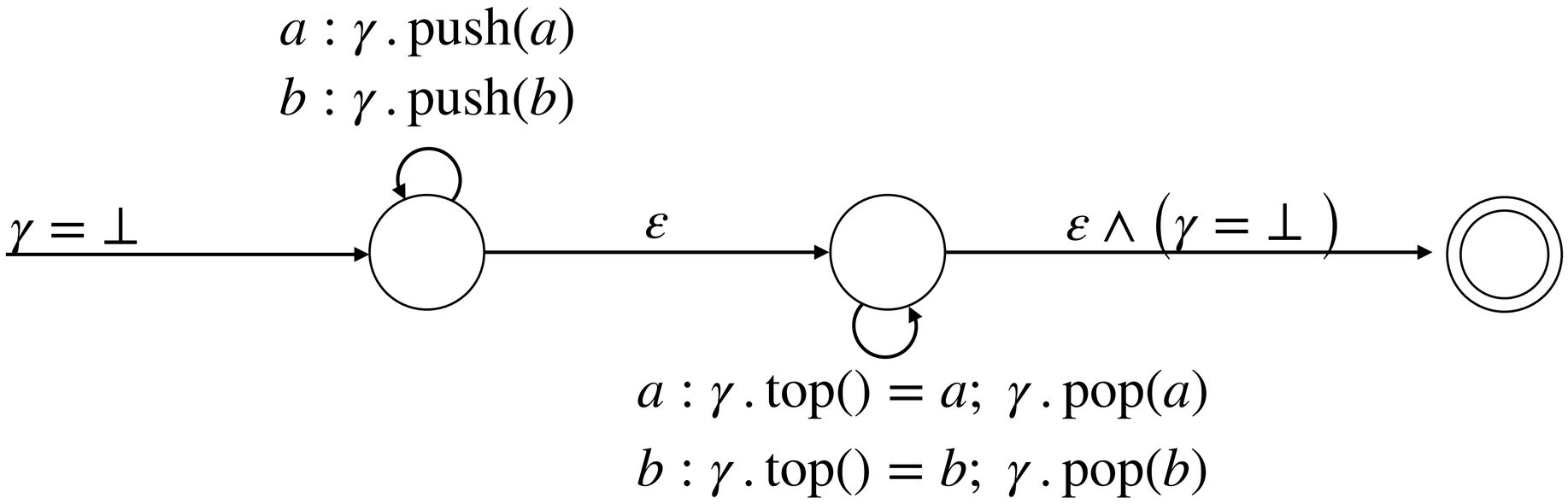
Theory of Computation

Lecture 9

Push-down Automata (PDA)

Consider $\{ww^R : w \in \Sigma^*\}$

We need to remember the count as well as what we saw.



Push-down Automata (PDA)

A (nondeterministic) pushdown automaton is a 7-tuple:

$$A = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. Γ is a finite stack alphabet.
4. δ is the transition function.
5. $q_0 \in Q$ is the start state.
6. $\perp \in \Gamma$ is the initial (bottom) stack symbol.
7. $F \subseteq Q$ is the set of accepting states.

Push-down Automata (PDA)

A (nondeterministic) pushdown automaton is a 7-tuple:

$$A = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$$

$$\delta : Q \times \left(\Sigma \cup \{\varepsilon\}\right) \times \Gamma \to \mathscr{P}\left(Q \times \Gamma^*\right)$$

If $(q', \alpha) \in \delta(q, a, X)$, then:

In state q, on reading a symbol $a \in \Sigma$ or ε, with $X \in \Gamma$on top of the stack, the machine may:

Move to state q' and replace X by $\alpha \in \Gamma^*$.

Dyck Languages

Let $\Sigma = \{[,]\}$. The Dyck language is defined as:

$\{u \in \Sigma^* :$ all prefixes of u contain no more]'s than
['s and the number of ['s in u equals the number of]'s$\}$

$$S \to \varepsilon \mid [S] \mid SS$$

Dyck-3 Language

Let $\Sigma = \{[,], (,), \{, \}\}$. The Dyck language is defined as:

$$S \rightarrow \varepsilon \mid (S) \mid \{S\} \mid [S] \mid SS$$

... and similarly for arbitary k.

Dyck-3 Language

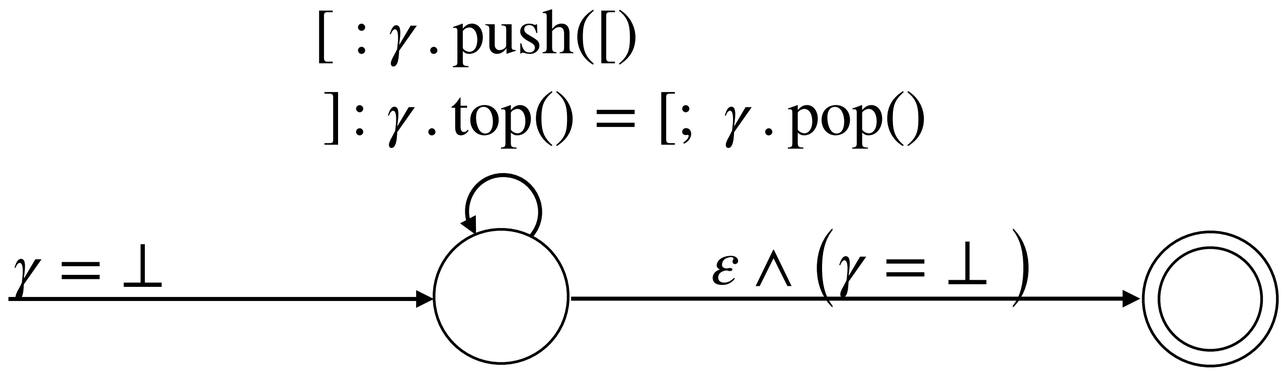
Let $\Sigma = \{[,], (,), \{, \}\}$. The Dyck language is defined as:

$$S \to \varepsilon \mid [S] \mid \{S\} \mid (S) \mid SS$$

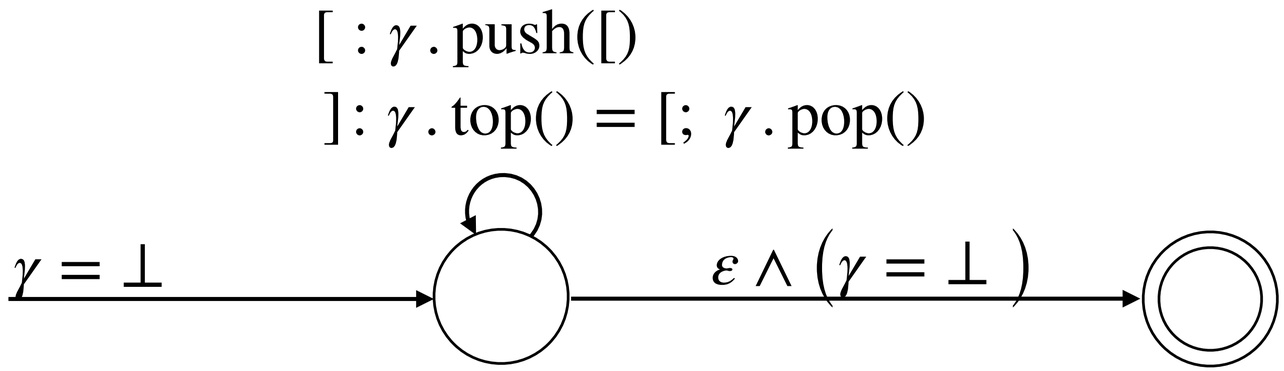
... and similarly for arbitary k.

Design a PDA for Dyck-3 language.

Dyck-1 Language



Dyck-3 Language



Dyck-3 Language

$(: \gamma.\text{push}(()$

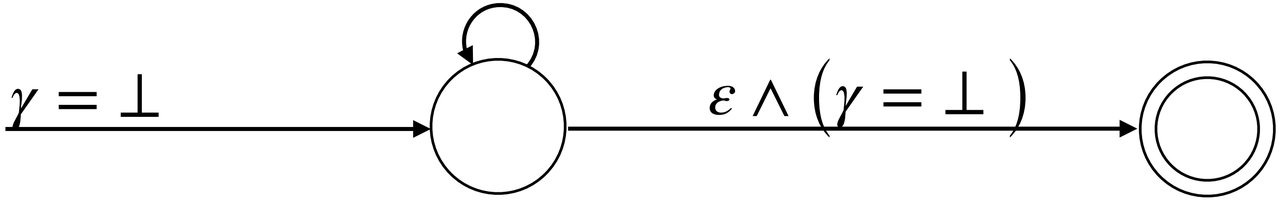
$) : \gamma.\text{top}() = (;\ \gamma.\text{pop}()$

$\{ : \gamma.\text{push}(\{)$

$\} : \gamma.\text{top}() = \{;\ \gamma.\text{pop}()$

$[: \gamma.\text{push}([)$

$] : \gamma.\text{top}() = [;\ \gamma.\text{pop}()$



Dyck-3 Language

$(: \gamma.\text{push}(1)$

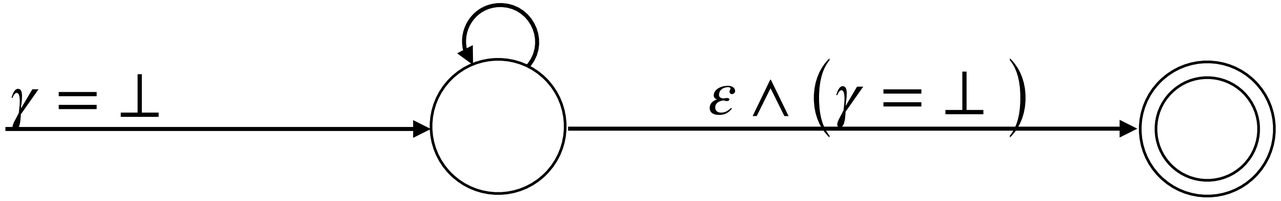
$) : \gamma.\text{top}() = 1;\ \gamma.\text{pop}()$

$\{ : \gamma.\text{push}(2)$

$\} : \gamma.\text{top}() = 2;\ \gamma.\text{pop}()$

$[: \gamma.\text{push}(3)$

$] : \gamma.\text{top}() = 3;\ \gamma.\text{pop}()$

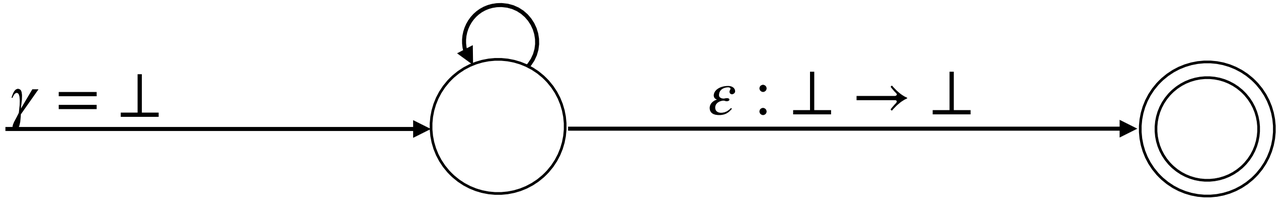


Dyck-3 Language

$$(: _ \to 1$$
$$) : 1 \to \varepsilon$$

$$\{ : _ \to 2$$
$$\} : 2 \to \varepsilon$$

$$[: _ \to 3$$
$$] : 3 \to \varepsilon$$

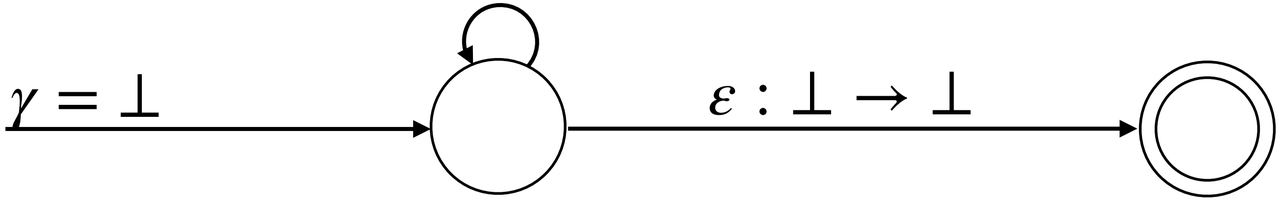


Dyck-3 Language

$$(: _ \to 1$$
$$) : 1 \to \varepsilon$$

$$\{ : _ \to 2$$
$$\} : 2 \to \varepsilon$$

$$[: _ \to 3$$
$$] : 3 \to \varepsilon$$



Show that $\{a^i b^j c^k : i = j \vee j = k \vee k = i\}$ is context-free.

Show that $\{a^i b^j c^k : i = j \vee j = k \vee k = i\}$ is context-free.

$$S \to S_1 \mid S_2 \mid S_3$$

$$S_1 \to S_1' C \qquad S_1' \to \varepsilon \mid a S_1' b \qquad A \to \varepsilon \mid aA$$

Show that $\{a^i b^j c^k : i = j \vee j = k \vee k = i\}$ is context-free.

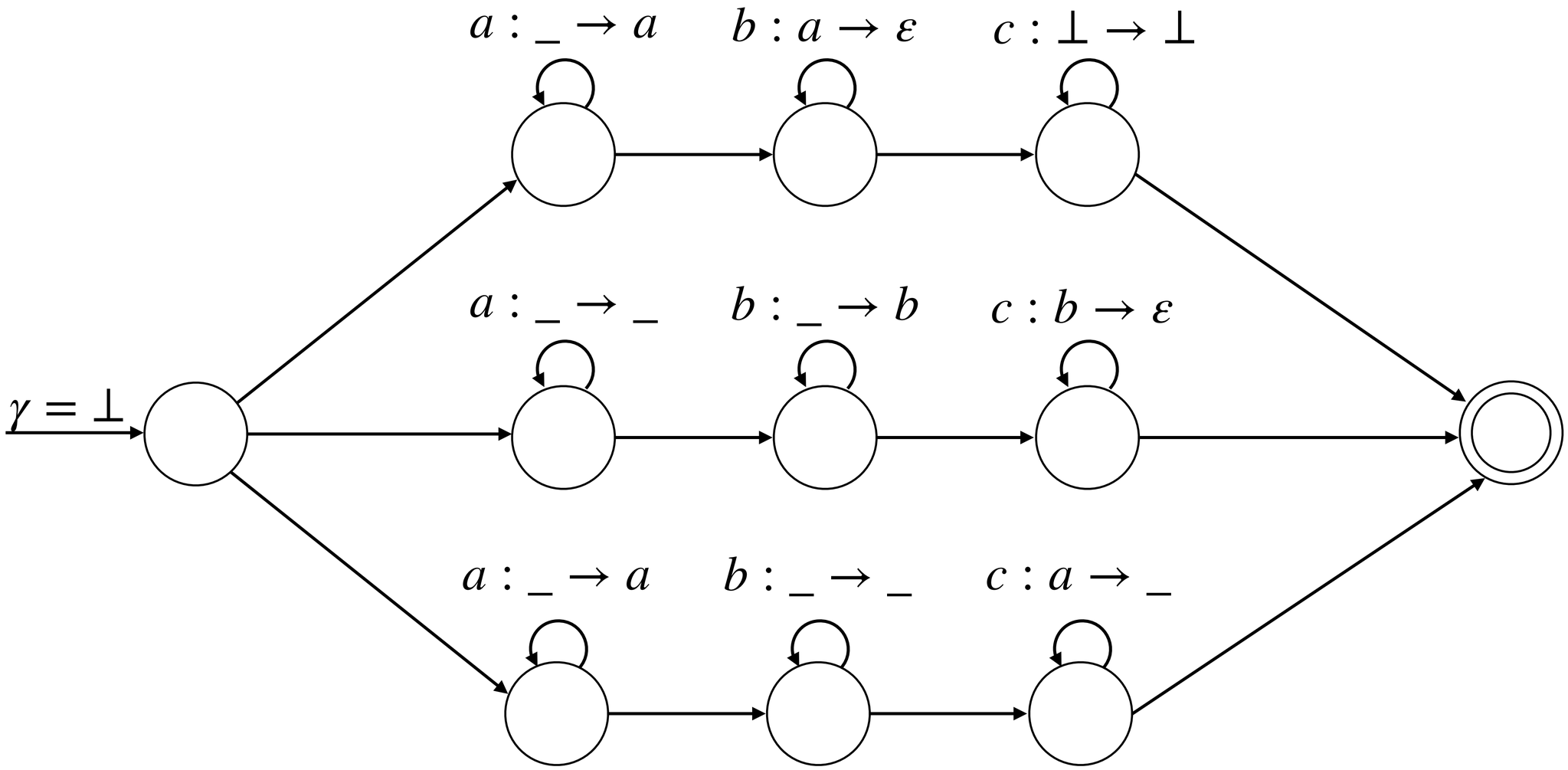
$$S \to S_1 \mid S_2 \mid S_3$$

$$S_1 \to S_1' C \qquad S_1' \to \varepsilon \mid aS_1'b \qquad A \to \varepsilon \mid aA$$

$$S_2 \to B \mid aS_2c \qquad B \to \varepsilon \mid bB$$

$$S_3 \to AS_3' \qquad S_3' \to \varepsilon \mid bS_3'c \qquad C \to \varepsilon \mid cC$$

Show that $\{a^i b^j c^k : i = j \vee j = k \vee k = i\}$ is context-free.



Automata with Other Data Structures

1. Counters (registers)
2. Boolean flags
3. Finite Stacks
4. Multiple Stacks

1. Queues
2. Lists
3. Arrays
4. Heaps

Multiple Stack Automata

$$\mathbf{1Stack} \subsetneq \mathbf{2Stack} = \mathbf{kStack}$$

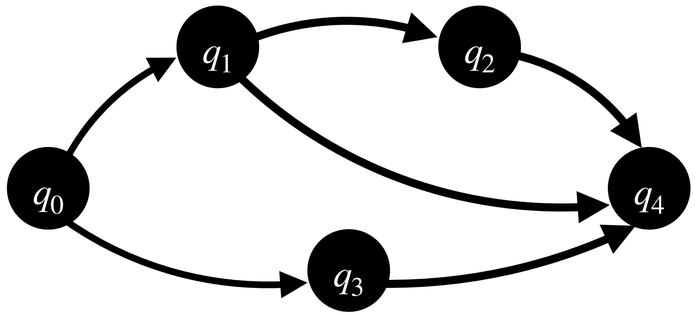
At any point, let the stacks be $(S_1, S_2, \cdots, S_k)$.

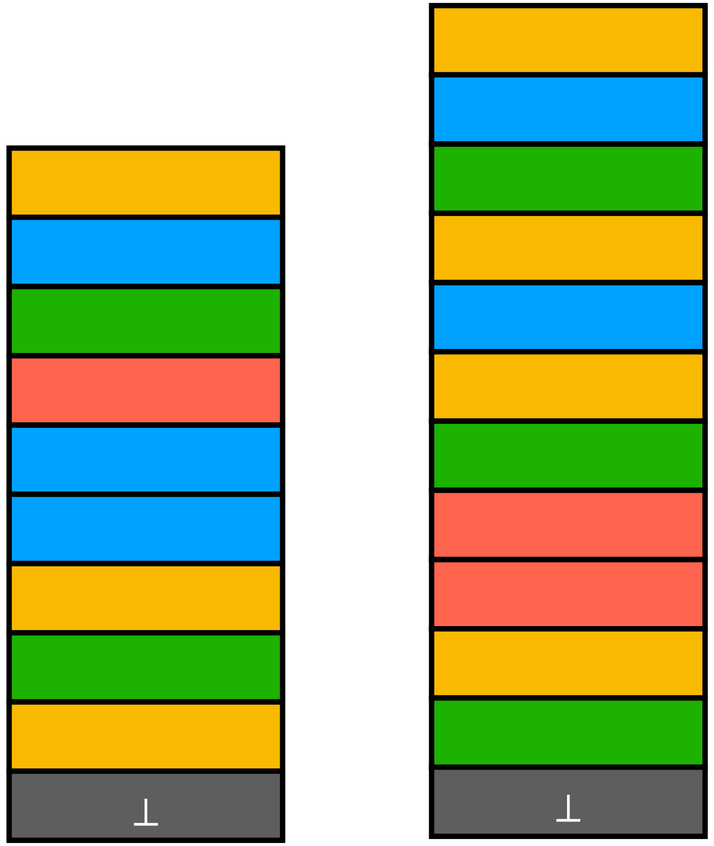
In one stack store $S_1\#S_2\#\cdots\#S_k$. Keep the other empty.

To update stack S_i, move all of
$S_1\#S_2\#\cdots\#S_{i-1}\#$ to the empty stack.

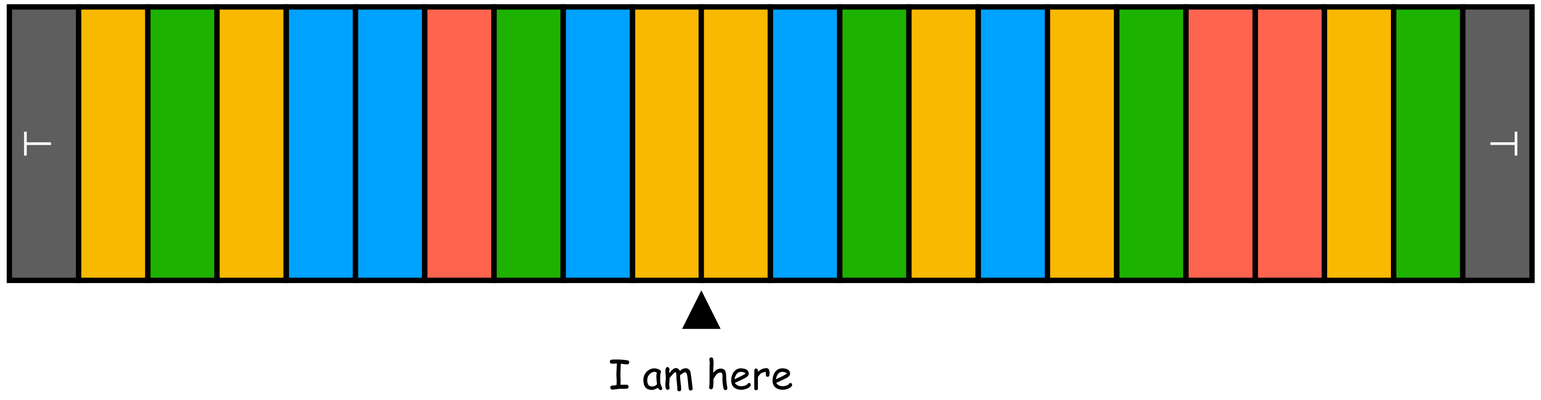
Update S_i, move all of $S_1\#S_2\#\cdots\#S_{i-1}\#$ back.

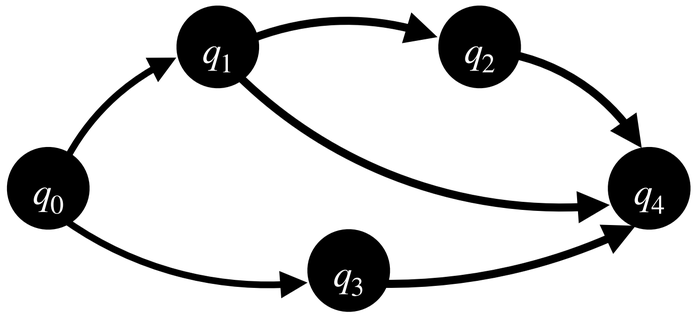
2-Stack Automata



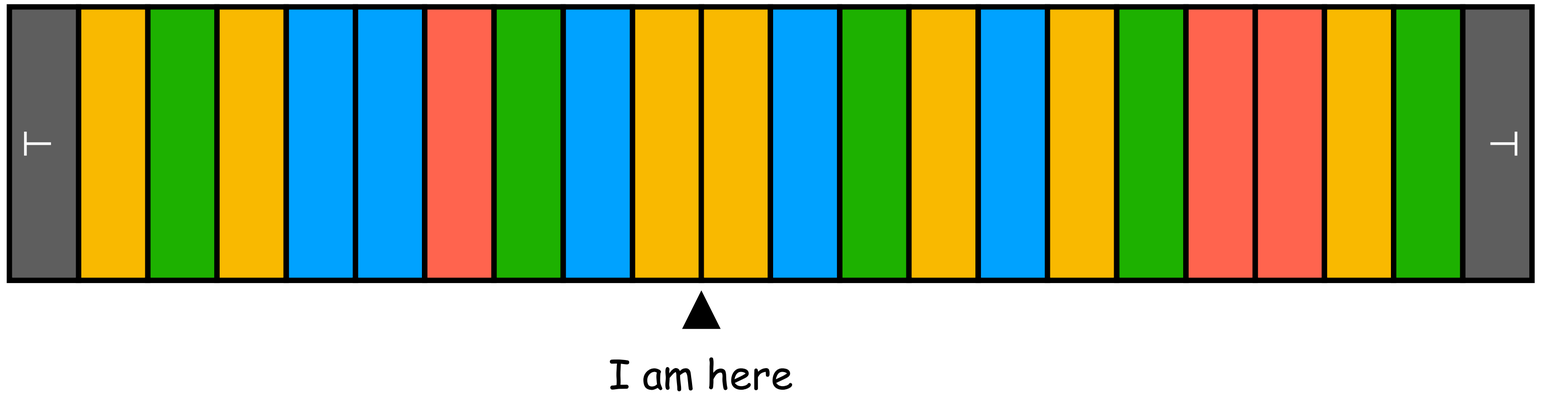


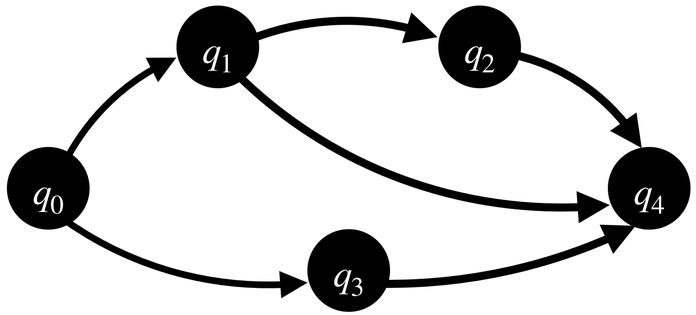
Tape Automata





Tape Automata





The machine has finite control states

It reads/writes symbols from an *infinite* tape.

The tape head can move left or right.

Tape Automata

A (deterministic) Tape automaton is a 9-tuple:

$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. $\Gamma \supset \Sigma$ is a finite stack alphabet.
4. $\vdash \in \Gamma \backslash \Sigma$ is a left endmarker
5. $\sqcup \in \Gamma \backslash \Sigma$ is a blank marker
6. δ is a transition function
7. s, t, and r are start, accept, and reject states.

Tape Automata

A (deterministic) Tape automaton is a 9-tuple:

$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

and

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

Intuitively, $\delta(q, a) = (q', b, d)$ means, when in state q scanning symbol a, write b on that tape cell, move the head in direction $d \in \{L, R\}$, and enter state q'.

Tape Automata

The tape is initialised with $\vdash w \sqcup^{\omega}$

We need additional constraints:

1. The left endmarker is never overwritten with another symbol.
2. The automaton never moves off the tape (to the left of the endmarker).
3. Once in accepting state, the automaton remains in accepting state.
4. When is reject state, the automaton remains in reject state.

$\vdash$	1	1	1	1	1	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

$\vdash$	1	1	1	1	1	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	1	1	1	1	1	1	1	$	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	1	1	1	1	1	1	$	0	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	1	1	1	1	1	$	0	0	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	1	1	1	1	1	$	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	1	1	1	$	0	0	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	1	1	1	$	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	0	0	1	$	0	0	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	0	0	1	$	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	0	0	0	$	0	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	0	0	0	0	$	1	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	1	1	1	1	1	1	1	$	1	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	1	1	1	1	1	1	$	0	1	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	1	1	1	1	1	$	0	0	1	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

Step 3: If m equals the input, move to t (accept state).

Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

$\vdash$	0	0	0	1	1	1	1	$	0	0	0	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$	$\sqcup$

Step 1: Write $ followed by the number 2 in unary. Denote it by m.

Loop:

Step 2: Remove occurrences of m from the input string.

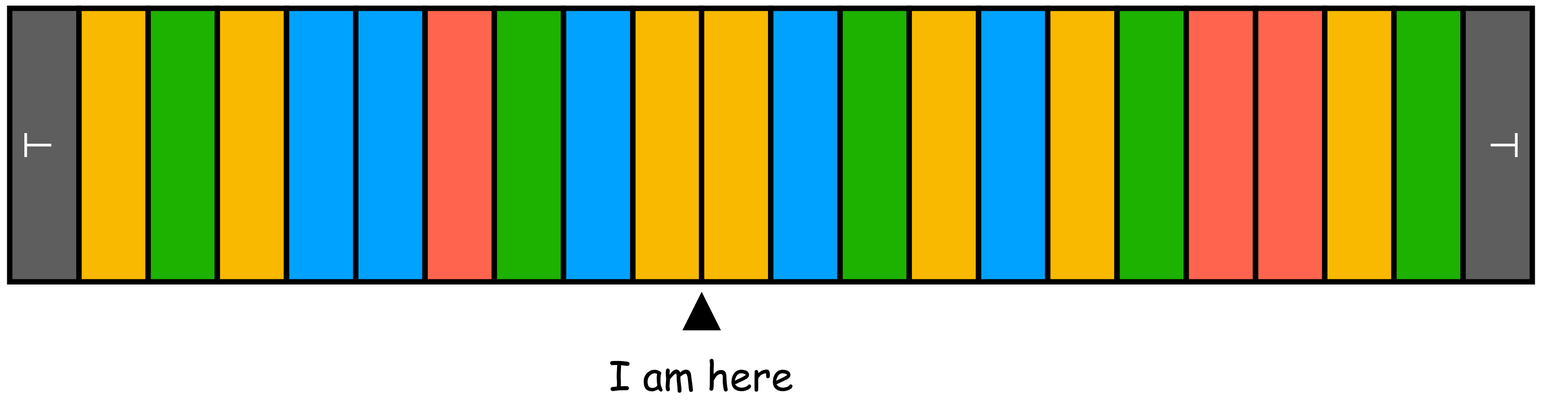
Step 3: If m equals the input, move to t (accept state).

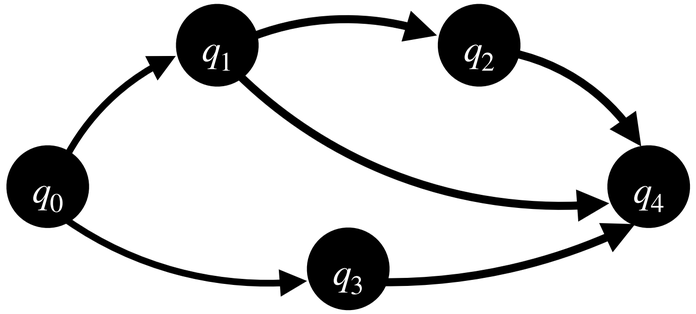
Step 4: If done evenly, move to r (reject state).

Step 5: If exceeded but in multiple tries, increment the number m.

Step 6: Else, reset the string and loop again.

What is this machine?

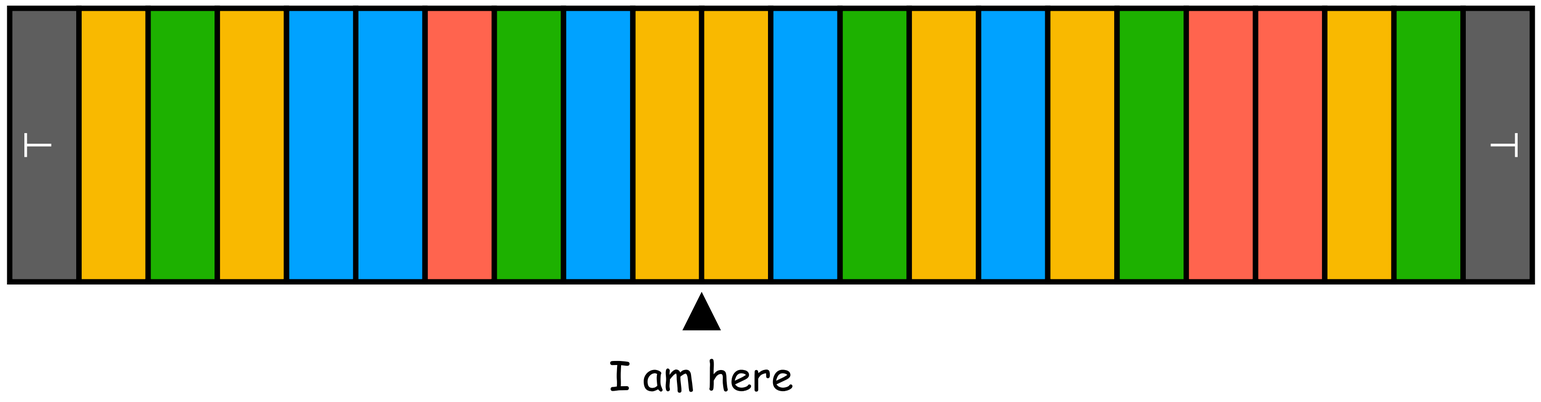


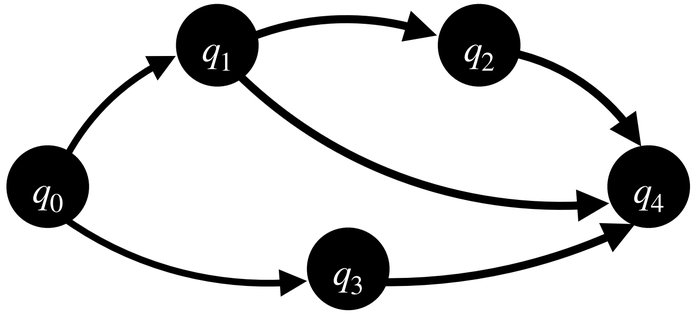


There are finitely many instructions.

There is infinite memory.

What is this machine?





There are finitely many instructions.

There is infinite memory.

Turing Machine

Configurations

At any point in time, the tape of the Turing machine contains a *semi-infinite* string of the form $\vdash w \sqcup^{\omega}$, where $w \in \Gamma^*$ is finite length.

Configurations

At any point in time, the tape of the Turing machine contains a *semi-infinite* string of the form $\vdash w \sqcup^{\omega}$, where $w \in \Gamma^*$ is finite length.

A configuration is a snapshot of all relevant information about the machine

Control State

Tape

Read/Write Head

Configurations

At any point in time, the tape of the Turing machine contains a *semi-infinite* string of the form $\vdash w \sqcup^{\omega}$, where $w \in \Gamma^*$ is finite length.

We can represent it by just $w \in \Gamma^*$.

A configuration is a snapshot of all relevant information about the machine

$$Q \times \Gamma^* \times \mathbb{N}$$

Configurations

At any point in time, the tape of the Turing machine contains a *semi-infinite* string of the form $\vdash w \sqcup^{\omega}$, where $w \in \Gamma^*$ is finite length.

We can represent it by just $w \in \Gamma^*$.

The start configuration on input $x \in \Sigma^*$ is $(s, x, 0)$.

Configurations

For a string $z \in \Gamma^*$, let z_n be the nth symbol of z if $n < |w|$ and $\sqcup$ otherwise.

Let $s_b^n(z)$ denote the string obtained from z by substituting $b \in \Gamma$ for z_n at position n.

If $n \geq |w|$, $\quad s_b^n(z) = z \cdot \left(\sqcup^{n-|z|} \right) \cdot b.$

Configurations

Let $s_b^n(z)$ denote the string obtained from z by substituting $b \in \Gamma$ for z_n at position n.

if $\delta(p, z_n) = (q, b, L)$, then:

$$(p, z, n) \to (q, s_b^n(z), n - 1)$$

if $\delta(p, z_n) = (q, b, R)$, then:

$$(p, z, n) \to (q, s_b^n(z), n + 1)$$

Configurations

Let $s_b^n(z)$ denote the string obtained from z by substituting $b \in \Gamma$ for z_n at position n.

if $\delta(p, z_n) = (q, b, L)$, then:

$$(p, z, n) \to (q, s_b^n(z), n - 1)$$

if $\delta(p, z_n) = (q, b, R)$, then:

$$(p, z, n) \to (q, s_b^n(z), n + 1)$$

Define $\overset{*}{\to}$ as the reflexive transitive closure of $\to$.

Acceptance, Rejection, and Halting

A word w is **accepted** by a Turing Machine if:

There exists some $w' \in \Gamma^*$ and $n \in \mathbb{N}$ such that

$$(s, w, 0) \xrightarrow{*} (t, w', n)$$

Acceptance, Rejection, and Halting

A word w is **accepted** by a Turing Machine if:

$$(s, w, 0) \xrightarrow{*} (t, _, _)$$

Acceptance, Rejection, and Halting

A word w is **accepted** by a Turing Machine if:

$$(s, w, 0) \xrightarrow{*} (t, _, _)$$

A word w is **rejected** by a Turing Machine if:

$$(s, w, 0) \xrightarrow{*} (r, _, _)$$

Acceptance, Rejection, and Halting

A Turing Machine **halts** on an input w if it is accepted or rejected.

A Turing Machine is **total** if it halts on every input.

Languages

The set of words accepted by a Turing Machine is called the language of the machine.

$$\mathscr{L}(M) = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \xrightarrow{*} (t, w', n)\}$$

Languages

The set of words accepted by a Turing Machine is called the language of the machine.

$$\mathscr{L}(M) = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \xrightarrow{*} (t, w', n)\}$$

A language $L \subseteq \Sigma^*$ is **recursively enumerable** if there exists a Turing Machine M such that $L = \mathscr{L}(M)$.

Languages

The set of words accepted by a Turing Machine is called the language of the machine.

$$\mathscr{L}(M) = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \xrightarrow{*} (t, w', n)\}$$

A language $L \subseteq \Sigma^*$ is **recursively enumerable** if there exists a Turing Machine M such that $L = \mathscr{L}(M)$.

$$\mathrm{RE}(\Sigma)$$

Enumerator

An enumerator is a Turing machine

$$E = (Q, \Sigma, \#, \vdash, \delta, s)$$

with the following properties:

It starts on a blank tape (no input)

It has a distinguished output tape.

It writes strings over Σ on the output tape, separated by a delimiter symbol #.

Enumerator

An enumerator is a Turing machine

$$E = (Q, \Sigma, \#, \vdash, \delta, s)$$

$$\mathscr{L}(E) = \{w : E \text{ eventually prints } w\}$$

Enumerator

An enumerator is a Turing machine

$$E = (Q, \Sigma, \#, \vdash, \delta, s)$$

$$\mathscr{L}(E) = \{w : E \text{ eventually prints } w\}$$

$$L \in \mathrm{RE}(\Sigma) \iff \exists \text{ enumerator } E \text{ such that } L = \mathscr{L}(E)$$

Enumerator

$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists$ enumerator $E \,.\, L = \mathscr{L}(E)$

Enumerator

$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists$ enumerator $E \,.\, L = \mathscr{L}(E)$

Consider some enumeration over Σ^*: $w_1, w_2, w_3, \ldots$

Run M on $w_1, w_2, w_3, \ldots$.

If accepted, enumerate.

Else, skip.

Why is this wrong?

Enumerator

$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists$ enumerator $E \,.\, L = \mathscr{L}(E)$

Simulate each machine in parallel?

Enumerator

$$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists \text{ enumerator } E \,.\, L = \mathscr{L}(E)$$

$(s, w_1, 0)$

Simulate each machine in parallel?

Enumerator

$$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists \text{ enumerator } E \,.\, L = \mathscr{L}(E)$$

$$(s, w_1, 0) \quad \rightarrow \quad (q, w_1', 1)$$

Simulate each machine in parallel?

Enumerator

$$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists \text{ enumerator } E \,.\, L = \mathscr{L}(E)$$

$$(s, w_1, 0) \quad \rightarrow \quad (q_{1,1}, w_{1,1}, n_{1,1})$$

$$(s, w_2, 0)$$

Simulate each machine in parallel?

Enumerator

$$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists \text{ enumerator } E \,.\, L = \mathscr{L}(E)$$

$$\begin{array}{ccccc}
(s, w_1, 0) & \rightarrow & (q_{1,1}, w_{1,1}, n_{1,1}) & \rightarrow & (q_{1,2}, w_{1,2}, n_{1,2}) \\
 & & (s, w_2, 0) & \rightarrow & (q_{2,1}, w_{2,1}, n_{2,1}) \\
 & & & & (s, w_3, 0)
\end{array}$$

Simulate each machine in parallel?

Enumerator

$$L \in \mathrm{RE}(\Sigma) \Rightarrow \exists \text{ enumerator } E \,.\, L = \mathscr{L}(E)$$

$$\begin{array}{ccccccccc}
(s, w_1, 0) & \rightarrow & (q_{1,1}, w_{1,1}, n_{1,1}) & \rightarrow & (q_{1,2}, w_{1,2}, n_{1,2}) & \rightarrow & (q_{1,3}, w_{1,3}, n_{1,3}) & \rightarrow & \cdots \\
 & & (s, w_2, 0) & \rightarrow & (q_{2,1}, w_{2,1}, n_{2,1}) & \rightarrow & (q_{2,2}, w_{2,2}, n_{2,2}) & \rightarrow & \cdots \\
 & & & & (s, w_3, 0) & \rightarrow & (q_{3,1}, w_{3,1}, n_{3,1}) & \rightarrow & \cdots \\
 & & & & & & (s, w_3, 0) & \rightarrow & \cdots
\end{array}$$

If any of the processes reach accept,
print the corresponding word.

Enumerator

$L \in \mathrm{RE}(\Sigma) \Leftarrow \exists$ enumerator $E \,.\, L = \mathscr{L}(E)$

On input w,

1. Run E
2. For each x printed by E, check if $x = w$.
3. If so, accept, else continue.

Languages

The set of words accepted by a Turing Machine is called the language of the machine.

$$\mathscr{L}(M) = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \xrightarrow{*} (t, w', n)\}$$

A language $L \subseteq \Sigma^*$ is **recursively enumerable** if there exists a Turing Machine M such that $L = \mathscr{L}(M)$.

$$\mathrm{RE}(\Sigma)$$

Languages

The set of words not accepted by a Turing Machine ...

$$\overline{\mathscr{L}(M)} = \{w \in \Sigma^* : \exists w', n \,.\, (s, w, 0) \overset{*}{\not\to} (t, w', n)\}$$

A language $L \subseteq \Sigma^*$ is **co-recursively enumerable** if there exists a Turing Machine M such that $L = \overline{\mathscr{L}(M)}$.

$$\mathrm{coRE}(\Sigma)$$

Important Question:

Is $\mathrm{RE}(\Sigma) = \mathrm{coRE}(\Sigma)$?

That is, are recursively enumerable languages closed under complement?

Ashoka School of
Advanced Computing

February 26, 2026 | 5:00 p.m.

Computer Science: Webinar

Vibe Craft - Code & Build

Vibe Coding Workshop: An introduction to flow-based, tool-assisted, rapid programming

>> Webinar

Anand S

LLM Psychologist,
Straive,
Singapore

Human Behaviour is Deterministic

Is determinism consistent with the uncertainity principle?

Is there a “Human Behaviour?”

Human Behaviour is Deterministic

Given initial conditions, is the behaviour computable?

If there is no free will, what is individual responsibility?

A Computational Process is Correct

There is a *specification* and there is a proof that the process *satisfies* the specification.

There is a *distribution* over the inputs, and the process performs well in *expectation*.

A Computational Process is Correct

A concrete *implementation* is correct if it is *reliable* and *efficient* given hardware and resource constraints.

The program *looks* plausible and produces outputs that seem correct on the inputs I tried.

Multi-tape Turing Machines

A (deterministic) k-tape Turing Machine is a 9-tuple:

$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

where:

1. Q is a finite set of states.
2. Σ is a finite input alphabet.
3. $\Gamma \supset \Sigma$ is a finite stack alphabet.
4. $\vdash \in \Gamma \backslash \Sigma$ is a left endmarker
5. $\sqcup \in \Gamma \backslash \Sigma$ is a blank marker
6. δ is a transition function
7. s, t, and r are start, accept, and reject states.

Multi-tape Turing Machines

A (deterministic) k-tape Turing Machine is a 9-tuple:

$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

and

$$\delta : Q \times \Gamma^n \to Q \times \Gamma^n \times \{L, R\}^n$$

Intuitively, the machine has k tapes and k heads. The machine reads k symbols simultaneously, writes k symbols, and makes k moves on the tapes.

Multi-tape Turing Machines

The configuration is of the form $(q, w_1, n_1, w_2, n_2, \ldots, w_k, n_k)$.

Each (w_i, n_i) can be rewritten as $w_i' = u_i \$ a_i v_i$ where $|u_i| = n_i$, $\$$ is a fresh symbol and a_i is the symbol under the head.

Multi-tape Turing Machines

The configuration is of the form $(q, w_1, n_1, w_2, n_2, \ldots, w_k, n_k)$.

Each (w_i, n_i) can be rewritten as $w_i' = u_i \$ a_i v_i$ where $|u_i| = n_i$, $\$$ is a fresh symbol and a_i is the symbol under the head.

The configuration is of the form $(q, w_1', w_2', \cdots, w_k')$.

Multi-tape Turing Machines

To simulate a multi-tape TM, consider the corresponding configuration:

$$(q, w_1'\#w_2'\#\cdots\#w_k')$$

Multi-tape Turing Machines

To simulate a multi-tape TM, consider the corresponding configuration:

$$(q, w_1'\#w_2'\#\cdots\#w_k')$$

For each step of the k-tape machine, the single-tape machine:

- Scans entire tape
- Locates all marked symbols
- Stores them in finite control

Multi-tape Turing Machines

To simulate a multi-tape TM, consider the corresponding configuration:

$$(q, w_1'\#w_2'\#\cdots\#w_k')$$

For each step of the k-tape machine, the single-tape machine:

Using the stored symbols, apply the transition function.

Multi-tape Turing Machines

To simulate a multi-tape TM, consider the corresponding configuration:

$$(q, w_1'\#w_2'\#\cdots\#w_k')$$

For each step of the k-tape machine, the single-tape machine:

Using the stored symbols, apply the transition function.

- Replace marked symbols with new symbols
- Move head markers left or right

Multi-tape Turing Machines

If the k-tape TM runs in time $T(|w|)$,
then the single tape TM runs in time

$$O\left(T(|w|)^2\right)$$

Why?

Non-deterministic Turing Machines

Non-deterministic Turing Machines

A non-deterministic Turing Machine is a 9-tuple:

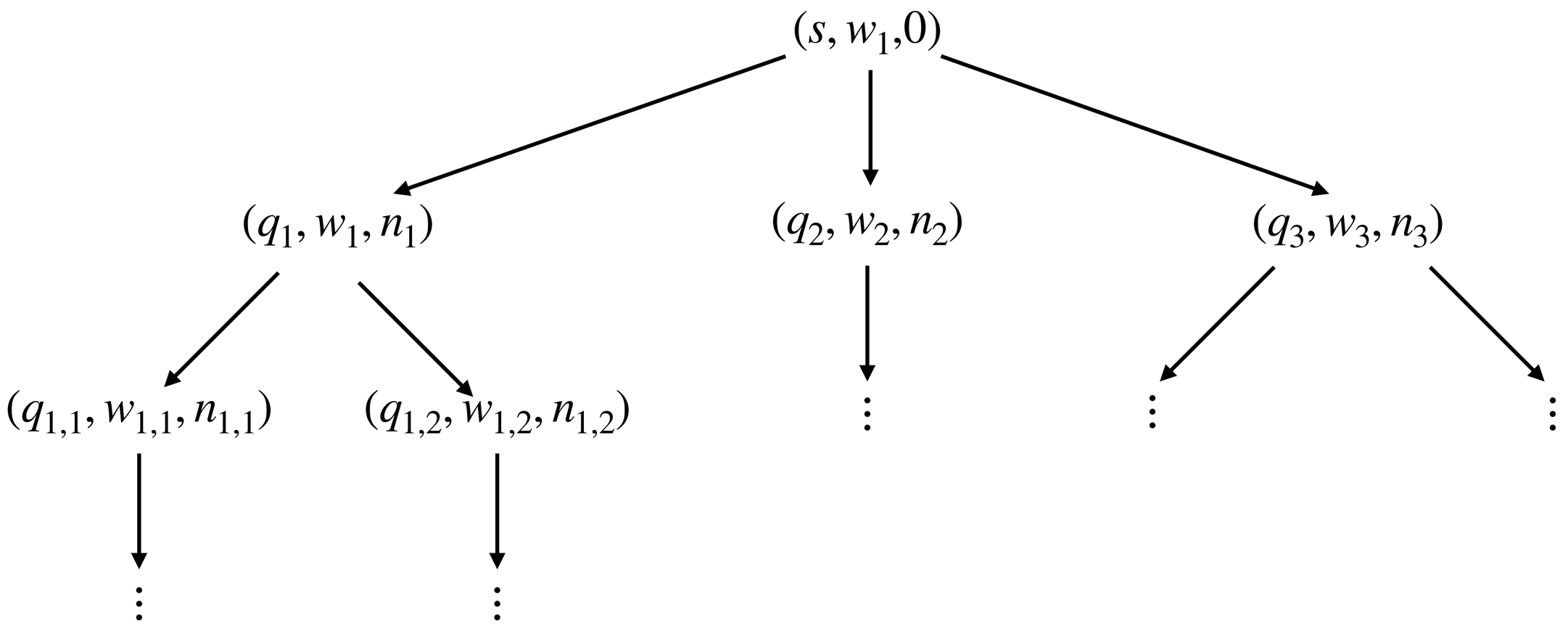
$$M = (Q, \Sigma, \Gamma, \vdash, \sqcup, \delta, s, t, r),$$

and

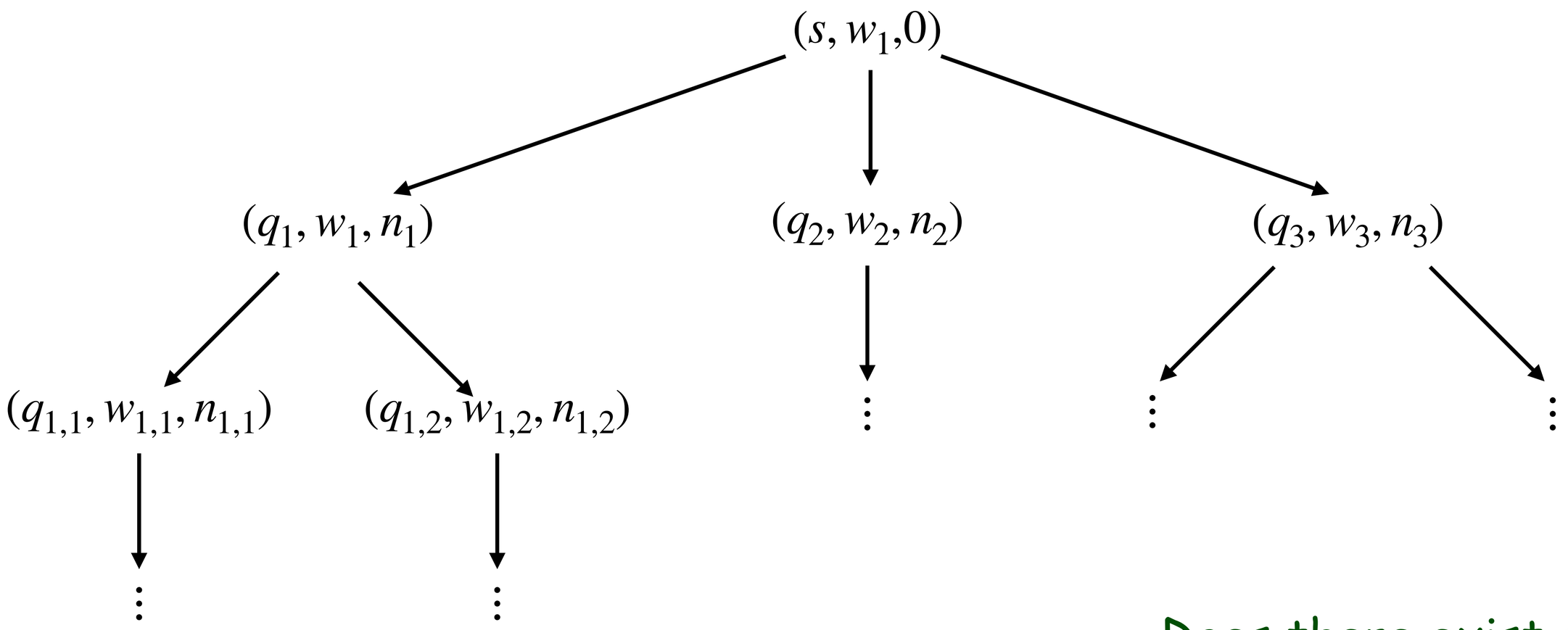
$$\Delta : Q \times \Gamma \to \mathscr{P}\left(Q \times \Gamma \times \{L, R\}\right)$$

This is analogous to an NFA. A word is accepted if there is an *accepting run*.

Non-deterministic Turing Machines

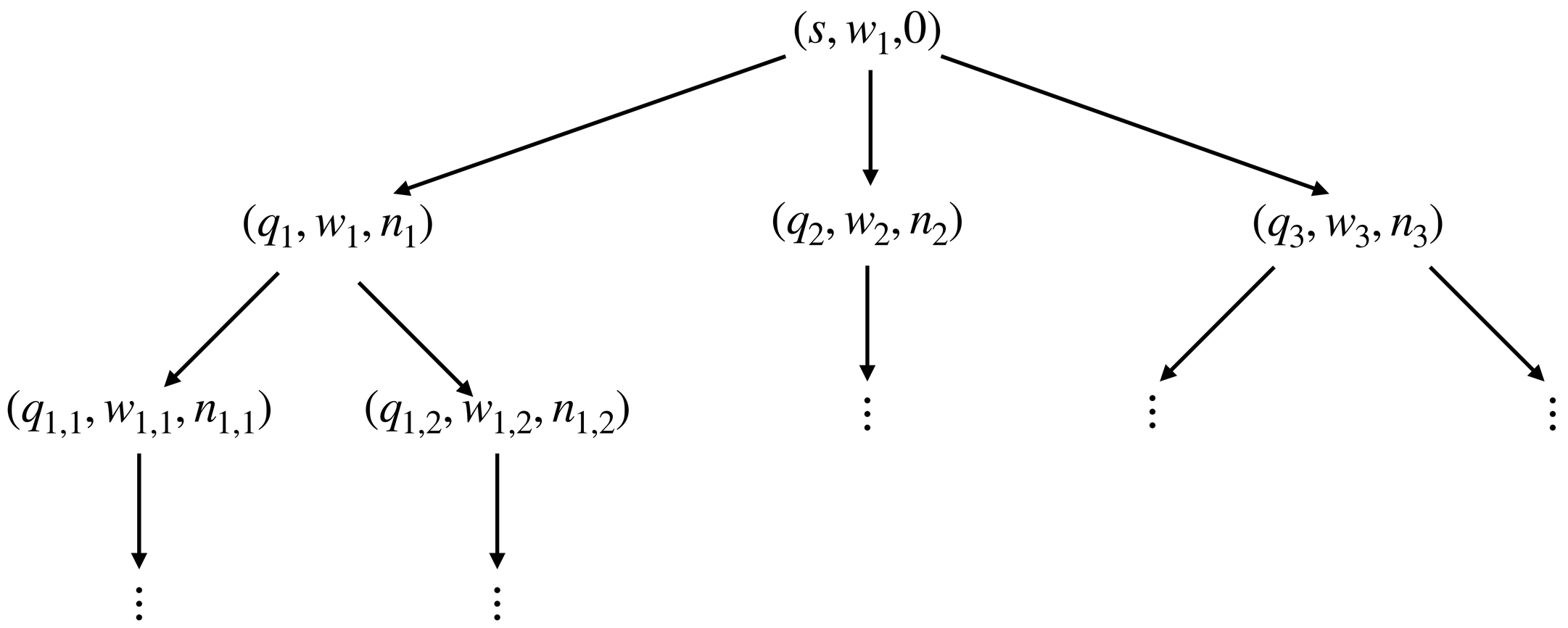


Non-deterministic Turing Machines



Does there exist a path to some $(t, _, _)$?

Non-deterministic Turing Machines



BFS!

Non-deterministic Turing Machines

Simulate it using a deterministic Turing Machine:

1. Encode configurations as strings
2. Maintain a queue of configurations
3. Repeatedly:
   (a) Dequeue one configuration
   (b) Generate all possible next configurations
   (c) Add them to the queue
   (d) If any configuration reaches accepting state, then accept

Non-deterministic Turing Machines

If the non-deterministic TM runs in time $T(|w|)$, and each state can lead to at most $k \leq 2 \times |Q \times \Gamma|$ configurations, then the deterministic TM runs in

Non-deterministic Turing Machines

If the non-deterministic TM runs in time $T(|w|)$, and each state can lead to at most $k \leq 2 \times |Q \times \Gamma|$ configurations, then the deterministic TM runs in

$$O\left(k^{T(|w|)}\right)$$

Can we do better?